

CodeFlash — 要件定義書

1. プロダクト概要

項目	内容
アプリ名（仮）	CodeFlash
コンセプト	プログラマー向け Notion風エディタ搭載フラッシュカードアプリ
ターゲットユーザー	CS学生・プログラミング学習者
モバイルアプリ	iOS / Android（Expo）
Web版	カード作成・編集専用（学習機能はモバイルのみ）
データ同期	iCloud同期（Appleデバイス間） + JSONエクスポート（Android・Web連携）
収益モデル	買い切り
UI言語	日本語 / 英語（i18n対応）

2. 機能要件

2.1 カード管理

データ構造

デッキ（Deck）

└─ カード（Card）

├─ 表面（Front） ─ ブロックの配列

├─ 裏面（Back） ─ ブロックの配列

├─ メモ（Memo） ─ ブロックの配列

└─ タグ（Tags） ─ タグIDの配列

- シンプルな2階層構造（デッキ > カード）
- 出題形式は「表（質問）→ 裏（答え）」の基本形のみ
- 1枚のカードの表・裏・メモそれぞれに複数ブロックを配置可能
- メモ欄: 学習時には表示せず、編集画面やカード詳細で確認できる補足ノート
 - 用途: 解説、参考URL、自分用の覚え書き、間違いやすいポイントなど

タグ機能

- カードに複数のタグを付与可能
- タグはデッキを横断してグローバルに管理
- 用途例: (#配列), (#ソート), (#O(n²)), (#Python), (#面接頻出), (#要復習)
- タグによるフィルタリング・検索
- タグ指定での学習セッション（デッキを横断して特定タグのカードだけ学習）
- タグの色分け（任意）

対応ブロックタイプ

ブロック	説明	優先度
テキスト	Markdown記法（見出し・太字・斜体・リスト等）	MVP
コードブロック	シンタックスハイライト付き、言語はプルダウン選択	MVP
画像	ローカルから挿入、カード内に表示	MVP
コード実行ブロック	後述のコード実行機能と連携	v1.0

2.2 カードエディタ

- 入力方式: Markdown記法ベース
- コードブロックの挿入: Notion風プルダウンUI
 - エディタ内で①または専用ボタンからブロックタイプを選択
 - コードブロック選択後、言語をプルダウンから指定（JavaScript, TypeScript, Python等）
 - 言語選択後、専用のコード入力エリアが展開される
- テキストブロック: Markdown記法で入力
- 画像ブロック: ボタンからローカル画像を選択して挿入
- タグ入力: エディタ下部にタグ入力欄、既存タグの候補サジェスト付き
- 編集タブ: 表面 / 裏面 / メモ をタブで切替
- プレビュー機能: 編集/プレビュー切替

エディタUI イメージ



コード

[▼ Python]

← コードブロック

def bubble_sort(arr):

言語はプルダウン選択

for i in range(...):

...

[/ ブロックを追加]

← Notion風ブロック追加

タグ: [#ソート] [#Python] [+]

← タグ入力 (サジェスト付き)

[プレビュー]

[保存]

2.3 学習画面：全画面表示モード

学習時にカードを全画面で表示し、集中して学習できるモード。

全画面表示の仕様

- ステータスバー・ナビゲーションバーを非表示にしたイマーシブ表示
- カード内容を画面中央に大きく表示
- コードブロックはフォントサイズを自動調整して可読性を確保
- 全画面モードの ON/OFF トグル（通常学習画面からワンタップで切替）

Bluetoothキーボード操作

キー	動作
Space	表→裏の切替（カードフリップ）
→ (右矢印)	次のカードへ
← (左矢印)	前のカードへ
①	自己評価「もう一度」
②	自己評価「難しい」
③	自己評価「普通」
④	自己評価「簡単」
M	メモの表示/非表示トグル
R	コード実行（実行可能ブロックがある場合）

想定ユースケース

- iPad + Magic Keyboard での学習（デスク学習）
- iPhone + Bluetoothキーボードでのハンズフリー学習
- 電車内などではタッチ操作、自宅ではキーボード操作と使い分け

2.4 コード実行機能

対応言語

言語	実行方式（案）	優先度
JavaScript / TypeScript	WebView内サンドボックス実行	MVP
Python	Pyodide（WASM）による実行	v1.0
SQL	sql.js（WASM SQLite）による実行	将来
C / C++	WASM コンパイラ or 外部API	将来

実行環境の要件

- セキュリティ: サンドボックス化し、端末リソースへのアクセスを禁止
- タイムアウト: 実行時間制限（例: 5秒）
- 出力: stdout / stderr をカード内に表示
- 状態: 実行ごとにリセット（ステートレス）
- 将来の言語追加を見据えた拡張可能なアーキテクチャ

2.5 復習（間隔反復）アルゴリズム

推奨: SM-2ベースのカスタム実装

SM-2を推奨する理由:

- Ankiで広く実績があり、学習効果のエビデンスが豊富
- 実装がシンプルで、後から調整しやすい
- ユーザーがAnkiからの移行時に違和感が少ない

復習フロー

カード表示 → ユーザーが回答確認 → 自己評価（4段階）

[もう一度] [難しい] [普通] [簡単]

↓ ↓ ↓ ↓

即再出題 間隔短縮 通常間隔 間隔延長

学習セッションの対象選択

- デッキ単位での学習（従来どおり）
- タグ指定での学習（デッキを横断して特定タグのカードを学習）
- 複数タグのAND/OR条件指定

保持するデータ（カードごと）

- easeFactor（容易度係数）：初期値 2.5
- interval（次回までの日数）
- repetitions（連続正解回数）
- nextReviewDate（次回復習日）
- lastReviewDate（最終復習日）

2.6 デッキ共有機能

- デッキをJSON形式でエクスポート / インポート（タグ情報含む）
- 共有URL or ファイル共有で他ユーザーに渡す
- 将来的にはアプリ内マーケットプレイス（クラウド同期後）

MVP段階での共有フロー：

作成者: デッキ → JSONエクスポート → ファイル共有（AirDrop等）

受信者: JSONファイル受取 → アプリでインポート → ローカルに保存

2.7 データ保存・同期

ローカル保存

- 技術: expo-sqlite
- バックアップ: JSONエクスポートによる手動バックアップ

iCloud同期（Appleデバイス間）

- 対象: iPhone / iPad / Mac 間でデッキ・カード・タグ・復習データを自動同期
- 技術: CloudKit（react-native-cloudkit or ネイティブモジュール）
- 同期対象データ: デッキ、カード（表面・裏面・メモ）、タグ、復習履歴（SM-2データ）、

画像

- **コンフリクト解決:** 最終更新日時（updatedAt）ベースの last-write-wins 方式
- **オフライン対応:** オフライン時はローカルに蓄積し、オンライン復帰時に自動同期
- **Android:** iCloud非対応のため、JSONエクスポート/インポートで対応

同期フローの概要

[iPhone で編集]

↓ 自動同期

[iCloud (CloudKit)]

↓ 自動同期

[iPad / Mac で最新データを取得]

※ Android / Web → JSONエクスポート/インポートで手動連携

2.8 Web版（カード作成専用）

- **目的:** PCの大きな画面でカードを快適に作成・編集する
- **機能範囲:** カード作成・編集・プレビューのみ（学習機能は含まない）
- **技術:** Expo Web（同一コードベース）
- **データ連携:** JSONエクスポート → モバイルアプリにインポート
- **将来対応:** クラウド同期でリアルタイム連携

3. 非機能要件

3.1 パフォーマンス

- カードの表裏切替: 60fps のスムーズなアニメーション
- コードハイライト: 500行以下のコードを100ms以内にレンダリング
- コード実行: 結果を5秒以内に返す（タイムアウト）
- アプリ起動: 3秒以内
- iCloud同期: バックグラウンドで実行し、UIをブロックしない

3.2 オフライン対応

- すべての基本機能がオフラインで動作すること
- コード実行もWASMによりオフライン実行
- iCloud同期はオンライン復帰時に自動再開

3.3 アクセシビリティ

- ダークモード対応
- フォントサイズ変更可能
- スクリーンリーダー対応（基本レベル）
- 外部キーボードによる完全操作（学習画面）

3.4 国際化（i18n）

- 日本語・英語の2言語対応
- 端末の言語設定に応じた自動切替
- 言語ファイルの分離（追加言語に拡張可能）

4. 技術スタック（案）

レイヤー	技術	選定理由
フレームワーク	Expo (React Native)	iOS/Android + Web版を1つのコードベースで対応
ナビゲーション	Expo Router	ファイルベースルーティング
状態管理	Zustand	軽量・シンプル
ローカルDB	expo-sqlite	構造化データの永続化、SQLクエリが使える
iCloud同期	CloudKit（ネイティブモジュール）	Appleデバイス間のシームレス同期
Markdownレンダリング	react-native-markdown-display	カード表示時のMarkdownレンダリング
シンタックスハイライト	react-syntax-highlighter	コードブロックの表示
JS実行	react-native-webview (サンドボックス)	JS/TSのカード内実行
Python実行	Pyodide (WebView経由)	Pythonのオフライン実行
i18n	i18next + react-i18next	多言語対応
アニメーション	react-native-reanimated	カードフリップ等のアニメーション
キーボード入力	react-native-keyevent or カスタムフック	Bluetoothキーボード操作の検知

5. 画面構成

モバイルアプリ (iOS / Android)

[ホーム画面]

- デッキ一覧
 - 新規デッキ作成
 - デッキ詳細 (カード一覧)
 - カード追加/編集 (エディタ)
 - タブ: 表面 / 裏面 / メモ
 - タグ入力欄
 - カード削除
 - タグでフィルタ
 - デッキ設定 (名前変更・削除)
 - エクスポート / インポート

— [学習画面]

- 通常モード
 - カード表面表示
 - タップで裏面表示 (フリップアニメーション)
 - メモ表示ボタン
 - コード実行ボタン
 - 自己評価ボタン (もう一度 / 難しい / 普通 / 簡単)
- 全画面モード (← ワンタップで切替)
 - イマーシブ全画面表示
 - タッチ操作 + Bluetoothキーボード操作
 - Escで通常モードに戻る
- 学習対象選択
 - デッキ単位
 - タグ単位 (デッキ横断)

— [タグ管理画面]

- タグ一覧 (カード数表示)
- タグの作成・編集・削除
- タグ色の設定

— [統計画面]

- 今日の学習数
- 復習スケジュール (今後の予定カード数)
- 学習ストリーク

└─ [設定画面]
└─ 言語切替（日本語 / English）
└─ テーマ（ライト / ダーク）
└─ フォントサイズ
└─ iCloud同期 ON/OFF
└─ キーボードショートカット設定
└─ データ管理（バックアップ / リストア）

Web版（カード作成専用）

[Web エディタ]
└─
└─ デッキ一覧（サイドバー）
└─ カード一覧（中央パネル）
└─ タグでフィルタ
└─ カードエディタ（メインエリア）
└─ タブ: 表面 / 裏面 / メモ
└─ Notion風ブロックエディタ
└─ タグ入力欄
└─ リアルタイムプレビュー（右ペイン）
└─ エクスポート（JSON出力）

6. データモデル

decks テーブル

カラム	型	説明
id	TEXT (UUID)	主キー
name	TEXT	デッキ名
description	TEXT	デッキの説明
language	TEXT	主にどの言語向けか（任意）
cardCount	INTEGER	カード数（キャッシュ）
createdAt	TEXT (ISO8601)	作成日時
updatedAt	TEXT (ISO8601)	更新日時

cards テーブル

カラム	型	説明
id	TEXT (UUID)	主キー
deckId	TEXT	所属デッキID（外部キー）
frontContent	TEXT (JSON)	表面のブロック配列（JSON）
backContent	TEXT (JSON)	裏面のブロック配列（JSON）
memoContent	TEXT (JSON)	メモのブロック配列（JSON）、NULL許容
createdAt	TEXT (ISO8601)	作成日時
updatedAt	TEXT (ISO8601)	更新日時

tags テーブル

カラム	型	説明
id	TEXT (UUID)	主キー
name	TEXT	タグ名（ユニーク）
color	TEXT	タグの色コード（例: ■ #FF6B6B）、NULL許容
createdAt	TEXT (ISO8601)	作成日時

card_tags テーブル（多対多リレーション）

カラム	型	説明
cardId	TEXT	カードID（外部キー）
tagId	TEXT	タグID（外部キー）

※ 複合主キー: (cardId, tagId)

reviews テーブル（SM-2データ）

カラム	型	説明
cardId	TEXT	カードID（外部キー・主キー）
easeFactor	REAL	容易度係数（初期 2.5）
interval	INTEGER	次回までの日数

repetitions	INTEGER	連続正解回数
nextReviewDate	TEXT (ISO8601)	次回復習日
lastReviewDate	TEXT (ISO8601)	最終復習日

ブロックのJSON構造例

```
json
[
  {
    "type": "text",
    "content": "## バブルソートとは\n時間計算量は  $O(n^2)$  です。"
  },
  {
    "type": "code",
    "language": "python",
    "content": "def bubble_sort(arr):\n    for i in range(len(arr)):\n        for j in range(len(arr)-1-i):\n            if arr[j] > arr[j+1]:\n                arr[j], arr[j+1] = arr[j+1], arr[j]\n    return arr",
    "executable": true
  },
  {
    "type": "image",
    "uri": "local://images/bubble_sort_diagram.png",
    "alt": "バブルソートの図解"
  }
]
```

7. 開発フェーズ

Phase 1 – MVP（4～6週間）

- デッキ・カードのCRUD（メモ欄含む）
- タグ機能（作成・付与・フィルタ）
- カードエディタ（Markdown + Notion風コードブロック挿入UI）
- シンタックスハイライト表示
- カードフリップ（表 → 裏）
- SM-2による間隔反復
- ローカルデータ保存（expo-sqlite）
- 日本語/英語 UI

Phase 2 — v1.0 (+3〜4週間)

- 全画面学習モード + Bluetoothキーボード操作
- JavaScript / TypeScript コード実行 (WebViewサンドボックス)
- Python コード実行 (Pyodide)
- 画像ブロック対応
- タグ指定での学習セッション (デッキ横断)
- デッキのJSON エクスポート/インポート (タグ情報含む)
- 学習統計画面
- ダークモード

Phase 3 — v1.1 (+4〜6週間)

- iCloud同期 (iPhone / iPad / Mac 間)
- Web版 (カード作成専用エディタ) の構築
- アプリ内課金 (買い切り) 実装
- ストア申請 (App Store / Google Play)
- パフォーマンス最適化

Phase 4 — 将来構想

- SQL コード実行 (sql.js)
- C/C++ コード実行
- クラウド同期 (Supabase等) → Android / Web版とのリアルタイム連携
- デッキマーケットプレイス (共有・検索・ダウンロード)
- AI によるカード自動生成 (コード → フラッシュカード)
- コミュニティ機能

8. リスクと対策

リスク	影響度	対策
WASM実行がモバイルで重い	高	実行言語をJS優先にし、Pythonは遅延ロード
Notion風エディタの実装難度	高	MVP段階ではシンプルなブロック追加UIに留め、段階的に改善
expo-sqliteのデータ上限	低	画像は外部ファイル参照、DBにはメタデータのみ
ストア審査でコード実行が問題に	中	サンドボックスの安全性を明示、Apple審査ガイドライン事前確認

Ankiとの差別化不足	高	コード実行+モダンUI+モバイルファースト+iCloud同期を徹底
Web版⇔モバイル間のデータ連携	中	Apple間はiCloud、それ以外はJSON連携、将来クラウド同期で統一
iCloud同期のコンフリクト	中	last-write-wins方式、将来的にマージUIを検討
CloudKitのExpo対応	高	ネイティブモジュール（Swift/Kotlin）で実装、Expo Config Pluginで統合
Bluetoothキーボードの機種差	低	主要キーのみ対応、設定画面でカスタマイズ可能に

9. 補足：買い切りモデルの設計案

プラン	内容
無料版	デッキ3つまで、カード各50枚まで、コード実行はJSのみ、iCloud同期なし
Pro（買い切り）	無制限デッキ・カード、全言語実行、エクスポート/インポート、統計機能、iCloud同期、全画面モード、Web版アクセス

想定価格帯: ¥980～¥1,480（類似アプリの相場を考慮）